

Solving the maze on its region tree

a note beside the APP1 report

Mohamed Ali Tewfik Hamilil
M1 Artificial Intelligence, Université Grenoble Alpes

September 12, 2026

Abstract

The report answers Problem 2 with Dijkstra’s algorithm on the maze graph, which settles 83–89% of the cells at every size measured. That is more than the problem requires. The generator already builds a binary tree of regions, and that tree decides where the route goes without any cell being examined: a division contributes a door to the route exactly when it separates the two endpoints. Reading the route off the tree costs $\Theta(n^{0.68})$ node visits against Dijkstra’s $\Theta(n)$, and $O(\log n)$ extra memory against $\Theta(n)$. This note states the lemma, gives the algorithm, derives the exponent from a branching recurrence, and measures the one parameter that recurrence needs.

1 The question

Every search in the report settles a large fraction of the maze. At 256×256 cells Dijkstra settles 54 293 of 65 536, and A* with the Manhattan estimate settles 52 085 — a saving of 4% on a problem whose answer is a route of about 4 300 steps.

That is the wrong order of magnitude, and the reason is that the search is told nothing about how the maze was built.

The generator does not merely produce a maze. It produces a **binary tree of regions**, and that tree records exactly where the walls are. A search that reads it need not look at the maze at all.

2 The separation lemma

Write R for an internal node of the region tree, R_ℓ and R_r for its two children, and (u, v) for its door, with $u \in R_\ell$ and $v \in R_r$.

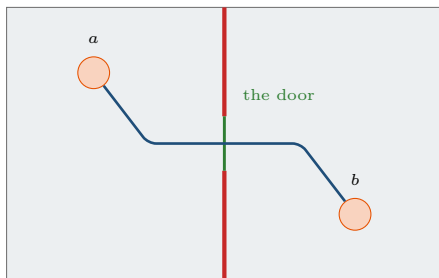
Proposition 1 (Separation). *Let a, b be cells of R . The unique route from a to b crosses the door (u, v) if and only if a and b lie in different children of R .*

Proof. The correctness proof of the generator establishes that (u, v) is the only edge of the maze between R_ℓ and R_r , and that each child is spanned by a tree of its own.

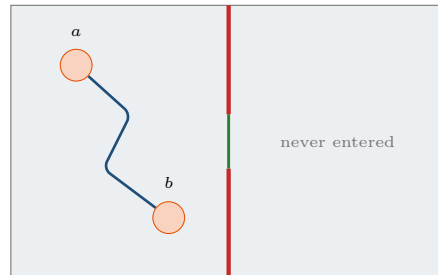
If a and b lie in different children, any route between them leaves one and enters the other, so it uses an edge between them; (u, v) is the only one.

If both lie in the same child, that child’s own spanning tree already contains a route between them. The maze is a tree, so its route between two cells is unique, and therefore that one. It stays inside the child and does not touch (u, v) . $\square$

Figure 1 is the two cases. The test is a comparison of coordinates against a rectangle — no cell is visited, no neighbour list is read.



(a) Opposite sides: the route *must* cross, and the door is the only crossing there is. Recurse on both halves.



(b) Same side: the route stays inside that half, and the other half is never looked at. Recurse once.

Figure 1: The separation lemma. Whether a division contributes a door to the route is decided by a containment test on its two children — no cell of the maze is examined. The right-hand case is where the saving comes from: half the region is discarded in one step.

3 The algorithm

Algorithm 1: The route between two cells, read off the region tree.

Input: a region-tree node R , and two cells a, b inside it.

Output: the number of doors between a and b (and, on the way, the route).

```

1 Function RouteInRegion( $R, a, b$ ):
2   if  $R$  is a corridor leaf then return  $|a_x - b_x| + |a_y - b_y|$ ;
3   let  $(u, v)$  be the door of  $R$ , with  $u \in R_\ell$  and  $v \in R_r$ ;
4   if Holds( $R_\ell, a$ ) and Holds( $R_\ell, b$ ) then return RouteInRegion( $R_\ell, a, b$ );
5   if Holds( $R_r, a$ ) and Holds( $R_r, b$ ) then return RouteInRegion( $R_r, a, b$ );
6   // separated: the door is on the route, by Proposition 1
7   if Holds( $R_\ell, a$ ) then return RouteInRegion( $R_\ell, a, u$ ) + 1 + RouteInRegion( $R_r, v, b$ );
8   return RouteInRegion( $R_r, a, v$ ) + 1 + RouteInRegion( $R_\ell, u, b$ );

```

Correctness is Proposition 1 applied at every node, with the corridor leaves as the base case: a corridor is a straight line, so the distance inside it is the difference of coordinates.

Two properties are worth naming.

A corridor costs one visit, not its length.

A leaf returns a run of cells as a single number. This is why the count below falls *below* the length of the route it computes.

The maze is never read.

Only the region tree is. The neighbour lists of Section 2 of the report are not consulted at all.

4 Why it is sublinear

At each division, one of two things happens.

- The endpoints are on the **same side**. One recursive call, and half the region is discarded unexamined.

- They are **separated**. Two recursive calls, one per half.

Write p for the probability of the second. On a region of m cells, each call doing $O(1)$ work of its own,

$$T(m) = (1 - p)T(m/2) + p \cdot 2T(m/2) + O(1) = (1 + p)T(m/2) + O(1),$$

and the master theorem gives

$$T(n) = \Theta\left(n^{\log_2(1+p)}\right).$$

The two endpoints of the scale are worth reading off. At $p = 1$ — every division separating them — the recurrence is $2T(n/2)$ and the cost is $\Theta(n)$, no better than Dijkstra. At $p = 0$ it is $T(n/2)$ and the cost is $\Theta(\log n)$, a single root-to-leaf descent.

$\Theta(\log n)$ is therefore not available. It would require the endpoints never to be separated, which happens only when they share a corridor. $\log n$ is the cost of **one** descent, and the search pays one descent per separation.

5 Measuring p

The recurrence has one parameter, so it makes one prediction, and the prediction is checkable. Sixty mazes per size, counting at every visited division whether it recursed once or twice.

Endpoints	n	Visits	One-way	Two-way	p	$\log_2(1 + p)$
far corners	1024	115	26.7	43.9	0.62	0.70
far corners	16384	703	171.8	264.9	0.61	0.68
far corners	65536	1588	388.1	599.7	0.61	0.68
uniform random	1024	73	—	—	0.54	0.63
uniform random	16384	408	—	—	0.58	0.66
uniform random	65536	950	—	—	0.59	0.67

The exponent fitted to the measured visits is 0.66–0.72 for far corners, against the 0.68 the recurrence predicts from $p = 0.61$. The two agree at every size.

p is not a constant of nature: it depends on where the endpoints are. Corners are the hard case, since a division far from both is likelier to fall between them; uniformly random endpoints give $p \approx 0.58$ and a slightly smaller exponent. Both are comfortably below 1, which is what matters.

6 What it costs, beside the alternatives

Method	Visits at $n = 65536$	Time	Extra memory
Dijkstra	54 293	$\Theta(n \log n)$	$\Theta(n)$
A*, Manhattan	52 085	$\Theta(n \log n)$	$\Theta(n)$
Region tree	1 588	$\Theta(n^{0.68})$	$O(\log n)$
Binary-tree maze, greedy walk	510	$\Theta(\sqrt{n})$	$O(1)$

The memory column is the sharper difference. Dijkstra needs a distance, a parent and a settled flag for every cell before it starts. The region-tree search needs only its own recursion stack, which is the depth of the tree.

The last row is a different maze, not a different search: the binary-tree generator makes the Manhattan distance *exact*, so the route is found by walking north-or-west with no search at all.

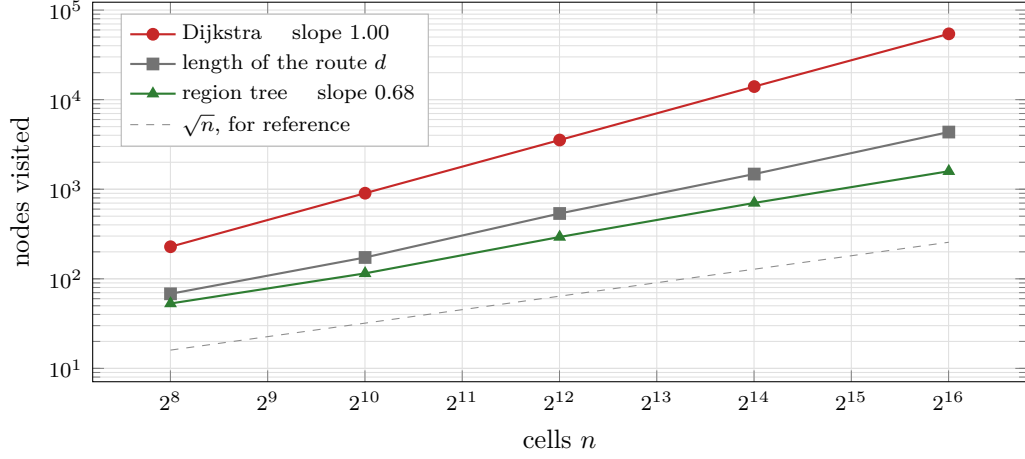


Figure 2: Both axes logarithmic, so a power law is a straight line and the exponent is its slope. Dijkstra runs parallel to n ; the region-tree search runs at slope 0.68, below even the length of the route it returns, because one visit to a corridor leaf covers a whole straight run.

It is listed to show where the scale ends — and what it costs, since a maze solvable by always turning the same way is barely a maze.

7 What this would change in the report

Section 2.8 of the report lists the region tree as feeding the cost analysis only. On the evidence here it should feed the search as well, and the handout's own learning objectives point that way: they ask for traversals of a binary tree and for *searching a path linking two nodes in a binary tree*, which is Algorithm 1 and not Dijkstra.

Two things would have to be said carefully if it were folded in.

- The exponent 0.68 is measured, and derived from a p that is itself measured. It is not a worst-case bound: the worst case is $\Theta(n)$, when every division separates the endpoints.
- Returning the *distance* costs $\Theta(n^{0.68})$; writing out the route cell by cell costs $\Theta(d)$ on top, because that is the size of the output. The compression is in the corridors, and it survives only as long as the answer stays compressed.